Nutrition Rules v9 Implementation Audit

Generated August 12, 2026. Scope: backend calculation pipeline, ready-meal filtering and ranking, swap/rebalance validation, browser-side option checks, and regression tests.

Result

All nutrition-coaching rules in docs/nutrition_coaching_rules_v9.md are implemented or verified. The main corrected behavior was Section 8/9: meal windows now store only calories, protein, and fat; carbs are computed dynamically from each candidate meal actual protein and fat. No fallback relaxation is used when a slot has no valid ready-meal option.

Area	Status	Evidence
BMR/TDEE and goals	Verified	Mifflin-St Jeor, activity factors, loss deficit, maintenance, and fixed gain surplus match v9.
Daily macros	Verified	Protein range 1.8-2.2 g/kg, fat range 0.66-1.0 g/kg, default 2.0/0.7 g/kg, carbs fill remaining calories.
Meal distributions	Verified	All meal count x pattern tables sum to 100%; snacks remain unchanged in heavy patterns.
Meal macro windows	Corrected	Raw database ratio table plus 20% protein floor, scaled separately for min and max, sums exactly to daily protein/fat ranges.
Candidate filtering	Corrected	Calories, scaled protein/fat, and dynamic carb constraints are all inclusive hard filters.
Swaps	Verified	Swaps reuse only the selected slot filtered set; other slot windows are unchanged.

End-to-End Flow

1. Client body data is validated: weight, height, age, sex, activity level, goal, meal count, meal distribution, diet type, and optional macro preferences must be in the allowed v9 ranges.

2. BMR is calculated with Mifflin-St Jeor. Maintenance calories are BMR multiplied by the selected activity factor.

3. The goal adjusts maintenance calories: maintain is unchanged, lose weight subtracts a daily deficit from 0.5%-1.0% weekly body-weight loss, and gain weight adds a fixed 200-300 kcal surplus.

4. Daily macro targets are built protein first, fat second, and carbs last. The app keeps daily protein and fat ranges from body weight, and derives daily carb range from remaining calories.

5. Calories are split by the selected meal pattern. Each slot receives one calorie target and a +/-5% calorie window.

6. Each meal type uses the fixed database-derived ratio table. Breakfast and snack raw protein lows stay in the table, but the 20% protein floor is applied when building slot profiles.

7. Raw protein/fat gram windows are scaled so every meal window sum equals the daily protein/fat min and max. Infeasible slots stop generation with an explicit INFEASIBLE error.

8. Ready-meal candidates must pass all three constraints: slot calories, scaled protein/fat, and candidate-specific carb range. Passing candidates are ranked by calorie closeness, then protein midpoint, then fat midpoint.

9. Meal swaps and alternates reapply the same slot filter only. They do not rescale other meals and cannot introduce an out-of-window candidate.

Macro Windows Stored Per Meal

Each generated meal target now stores calories, proteinG, and fatG windows. It intentionally does not store macroWindows.carbG. For any candidate meal, carb bounds are computed as:

```
carb_min = (meal_calorie_min - candidate_protein * 4 - candidate_fat * 9) / 4
carb_max = (meal_calorie_max - candidate_protein * 4 - candidate_fat * 9) / 4
```

Testing Performed

Test command	Result
npm run check	Passed all JavaScript syntax checks.
npm run test:nutrition-v2	Passed 9/9 v9 audit sections: BMR/TDEE, goals, daily ranges, distributions, ratio table, scaling, filtering/ranking, swaps, UI wiring.
npm run test:nutrition-v9-massive	Passed 27,871 feasible client-plan windows, 6,689 intentional infeasible flags, 96,750 meal slots, and 193,500 boundary candidates.
node test-solver.js	Passed 22/22 solver, rebalance, exact-grid, and infeasible-bound checks.
npm run test:optimizer	Passed ready-meal optimizer matrix: 9/11 complete plans, 2 explicitly constrained as no valid option.
npm run test:deterministic-generation	Passed ready-meal generation tests.
npm run test:meal-chat-quality	Passed backend/chat frontend-disconnect quality checks.
npm run test:customers	Passed customer/dashboard/plan persistence tests.
npm test	Passed full suite end to end. Only observed runtime note was an existing pg SSL warning, not a test failure.

Files Updated

Core rule changes landed in src/config/nutritionConstants.js, src/services/nutritionService.js, and src/services/planGenerator.js. Browser-side option validation was updated in public/js/app.js. Tests were updated in scripts/testNutritionLogicV2.js, scripts/testNutritionV9Massive.js, scripts/testNutritionOptimizer.js, and scripts/testDeterministicMealGeneration.js. A small dashboard link/render cleanup and edit-mode copy cleanup were made so the existing full suite passes.

Operational Notes

When a client setup is mathematically infeasible, generation stops and flags the meal window instead of silently relaxing rules. When a ready-meal database slot has no candidate, the app surfaces an explicit no-valid-option state. Those outcomes are expected under v9 and were tested directly.